

Employee Management System

Requirements

1) The Company has employees

Employee has emp_code, email, phone, date_of_joining and status

2) Departments - Name, CostCenter, Current Manager

3) Each employee belongs to a department

Department membership must retain history of employee from and to

4) Roles define job function. Employee can hold multiple roles.

5) Compensation must be versioned

Basic, HRA, Allowances, Bonus

6) Attendance per employee with worktype and hours

Here WorkType is (Regular, Overtime, Leave, Work from home)

7) Leaves => Type, Start/End Date, Status, Approver

8) Assets => Laptop/Phone, Serial Number, issued date

9) Performance Reviews => Ratings, Comments, Submission Time

10 Access Groups => Employee added/Removed

This is a good real-world design question. I'll explain it in a **clean, beginner-friendly way**—focusing on **entities, relationships, and cardinality**, not just dumping tables.

☑ Step 1: Identify Main Entities

From the requirements, core entities are:

- Employee
- Department
- Role
- Compensation
- Attendance
- Leave
- Asset
- Performance Review
- Access Group

☑ Step 2: Understand Relationships (Most Important Part)

◇ 1. Employee ↔ Department

- Each employee belongs to a department
- BUT history must be stored (from_date, to_date)

👉 So:

- **Employee ↔ Department = Many-to-Many over time**
- Implement using a **history/junction table**

✓ Cardinality:

Employee → Department = M:N (via history)

◇ 2. Department → Manager (Employee)

- Each department has a **current manager**
- A manager is also an employee

👉 So:

- **Employee → Department = 1:N** (one manager can manage many departments)

◇ 3. Employee ↔ Role

- Employee can have multiple roles

👉 So:

Employee ↔ Role = Many-to-Many (M:N)

◇ 4. Employee → Compensation

- Compensation is **versioned**

👉 So:

- One employee → many compensation records (over time)

✓ Cardinality:

1 : N

◇ 5. Employee → Attendance

- Daily/periodic records

✓ Cardinality:

1 : N

◇ 6. Employee → Leave

- Employee applies leaves
- Leave has approver (another employee)

✓ Cardinality:

- Employee → Leave = **1:N**
- Approver (Employee) → Leave = **1:N**

◇ 7. Employee → Asset

- Employee can have multiple assets

✓ Cardinality:

1 : N

◇ 8. Employee → Performance Review

✓ Cardinality:

1 : N

◇ 9. Employee ↔ Access Group

- Employees added/removed

✓ Cardinality:

M : N

Step 3: Final Tables (Clean Schema)

◇ **Employee**

```
Employee(  
    emp_id PK,  
    emp_code,  
    email,  
    phone,  
    date_of_joining,  
    status  
)
```

◇ **Department**

```
Department(  
    dept_id PK,  
    name,  
    cost_center,  
    manager_id FK → Employee(emp_id)  
)
```

◇ **Employee_Department_History**

```
Emp_Department_History(  
    id PK,
```

```
    emp_id FK,  
    dept_id FK,  
    from_date,  
    to_date  
)
```

◇ Role

```
Role(  
    role_id PK,  
    role_name  
)
```

◇ Employee_Role

```
Employee_Role(  
    id PK,  
    emp_id FK,  
    role_id FK  
)
```

◇ Compensation (Versioned)

```
Compensation(  
    comp_id PK,  
    emp_id FK,  
    basic,  
    hra,  
    allowances,  
    bonus,  
    effective_from,  
    effective_to  
)
```

◇ Attendance

```
Attendance(  
    attendance_id PK,  
    emp_id FK,  
    date,  
    work_type,    -- Regular, Overtime, Leave, WFH  
    hours  
)
```

◇ Leave

```
Leave(  
    leave_id PK,  
    emp_id FK,  
    type,  
    start_date,  
    end_date,  
    status,  
    approver_id FK → Employee(emp_id)  
)
```

◇ Asset

```
Asset(  
    asset_id PK,  
    emp_id FK,  
    asset_type,    -- Laptop/Phone  
    serial_number,  
    issued_date  
)
```

◇ Performance Review

```
Performance_Review(  
  review_id PK,  
  emp_id FK,  
  rating,  
  comments,  
  submitted_at  
)
```

◇ Access Group

```
Access_Group(  
  group_id PK,  
  group_name  
)
```

◇ Employee_Access_Group

```
Employee_Access_Group(  
  id PK,  
  emp_id FK,  
  group_id FK,  
  added_at,  
  removed_at  
)
```

☑ Step 4: Cardinality Summary (VERY IMPORTANT)

Relationship	Cardinality
Employee → Department	M:N (with history)

Department → Manager	1:N
Employee → Role	M:N
Employee → Compensation	1:N
Employee → Attendance	1:N
Employee → Leave	1:N
Employee → Asset	1:N
Employee → Review	1:N
Employee → Access Group	M:N

Simple Way to Think (Fresher Trick)

Always ask:

👉 “Can one record have multiple on the other side?”

- Yes → Many
- No → One

Final Insight

- Use **M:N** when:
 - Multiple associations exist (Roles, Access Groups)
- Use **1:N** when:
 - One entity owns multiple records (Attendance, Assets)
- Use **history tables** when:
 - Data changes over time (Department, Compensation)

If you want, I can also:

- ✓ Draw **ER diagram (image)**
- ✓ Give **SQL with constraints + indexes**
- ✓ Convert this into **interview-ready answer format**

yes you can



Edit

Give SQL with constraints + indexes ☒ Convert this into interview-ready answer format

Alright — let's turn your design into a **clean, interview-ready answer** with:

- ☒ Proper SQL (with constraints)
- ☒ Indexes (important for performance)
- ☒ Structured explanation (so you can speak confidently)

☒ ☒ INTERVIEW-READY ANSWER

◇ 1. Approach (What you say first in interview)

“I identified core entities like Employee, Department, Role, Attendance, etc.
Then I normalized relationships using junction tables for M:N cases and added history/versioning tables where required (Department history, Compensation).”

☒ ◇ 2. SQL SCHEMA (WITH CONSTRAINTS)

◇ Employee

```
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_code VARCHAR(20) UNIQUE NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    phone VARCHAR(15),  
    date_of_joining DATE NOT NULL,  
    status VARCHAR(20) CHECK (status IN ('Active', 'Inactive'))  
);
```

◇ Department

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    cost_center VARCHAR(50),  
    manager_id INT,  
    FOREIGN KEY (manager_id) REFERENCES Employee(emp_id)  
);
```

◇ Employee Department History

```
CREATE TABLE Emp_Department_History (  
    id INT PRIMARY KEY,  
    emp_id INT NOT NULL,  
    dept_id INT NOT NULL,  
    from_date DATE NOT NULL,  
    to_date DATE,  
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id),  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);
```

◇ Role

```
CREATE TABLE Role (  
    role_id INT PRIMARY KEY,  
    role_name VARCHAR(50) UNIQUE NOT NULL  
);
```

◇ Employee Role (M:N)

```
CREATE TABLE Employee_Role (  
    id INT PRIMARY KEY,  
    emp_id INT NOT NULL,  
    role_id INT NOT NULL,  
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id),  
    FOREIGN KEY (role_id) REFERENCES Role(role_id),  
    UNIQUE (emp_id, role_id)  
);
```

◇ Compensation (Versioned)

```
CREATE TABLE Compensation (  
    comp_id INT PRIMARY KEY,  
    emp_id INT NOT NULL,  
    basic DECIMAL(10,2),  
    hra DECIMAL(10,2),  
    allowances DECIMAL(10,2),  
    bonus DECIMAL(10,2),  
    effective_from DATE NOT NULL,  
    effective_to DATE,  
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id)  
);
```

◇ Attendance

```
CREATE TABLE Attendance (  
    attendance_id INT PRIMARY KEY,  
    emp_id INT NOT NULL,  
    date DATE NOT NULL,  
    work_type VARCHAR(20) CHECK (work_type IN  
( 'Regular', 'Overtime', 'Leave', 'WFH' )),  
    hours DECIMAL(5,2),  
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id)  
);
```

◇ Leave

```
CREATE TABLE Leave_Request (  
    leave_id INT PRIMARY KEY,  
    emp_id INT NOT NULL,  
    type VARCHAR(50),  
    start_date DATE,  
    end_date DATE,  
    status VARCHAR(20),  
    approver_id INT,  
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id),  
    FOREIGN KEY (approver_id) REFERENCES Employee(emp_id)  
);
```

◇ Asset

```
CREATE TABLE Asset (  
    asset_id INT PRIMARY KEY,  
    emp_id INT,  
    asset_type VARCHAR(20),  
    serial_number VARCHAR(100) UNIQUE,  
    issued_date DATE,
```

```
FOREIGN KEY (emp_id) REFERENCES Employee(emp_id)
);
```

◇ Performance Review

```
CREATE TABLE Performance_Review (
  review_id INT PRIMARY KEY,
  emp_id INT NOT NULL,
  rating INT CHECK (rating BETWEEN 1 AND 5),
  comments TEXT,
  submitted_at TIMESTAMP,
  FOREIGN KEY (emp_id) REFERENCES Employee(emp_id)
);
```

◇ Access Group

```
CREATE TABLE Access_Group (
  group_id INT PRIMARY KEY,
  group_name VARCHAR(100) UNIQUE
);
```

◇ Employee Access Group (M:N + History)

```
CREATE TABLE Employee_Access_Group (
  id INT PRIMARY KEY,
  emp_id INT NOT NULL,
  group_id INT NOT NULL,
  added_at TIMESTAMP,
  removed_at TIMESTAMP,
  FOREIGN KEY (emp_id) REFERENCES Employee(emp_id),
  FOREIGN KEY (group_id) REFERENCES Access_Group(group_id)
);
```

3. INDEXES (VERY IMPORTANT FOR INTERVIEW)

```
CREATE INDEX idx_emp_dept_hist_emp ON Emp_Department_History(emp_id);  
CREATE INDEX idx_emp_dept_hist_dept ON  
Emp_Department_History(dept_id);
```

```
CREATE INDEX idx_employee_role_emp ON Employee_Role(emp_id);  
CREATE INDEX idx_employee_role_role ON Employee_Role(role_id);
```

```
CREATE INDEX idx_comp_emp ON Compensation(emp_id);
```

```
CREATE INDEX idx_attendance_emp_date ON Attendance(emp_id, date);
```

```
CREATE INDEX idx_leave_emp ON Leave_Request(emp_id);  
CREATE INDEX idx_leave_approver ON Leave_Request(approver_id);
```

```
CREATE INDEX idx_asset_emp ON Asset(emp_id);
```

```
CREATE INDEX idx_review_emp ON Performance_Review(emp_id);
```

```
CREATE INDEX idx_emp_group_emp ON Employee_Access_Group(emp_id);  
CREATE INDEX idx_emp_group_group ON Employee_Access_Group(group_id);
```

4. HOW TO EXPLAIN IN INTERVIEW (IMPORTANT)

You can say this:

“I used normalization to avoid redundancy.

- M:N relationships like Employee–Role and Employee–AccessGroup are handled via junction tables.

- Time-based changes like Department assignment and Compensation are handled using history/versioning tables.
- Constraints like CHECK, UNIQUE, and FK ensure data integrity.
- Indexes are added on foreign keys and frequently queried columns for performance.”



PRO TIPS (Interview Gold)

- Always mention **why**:
 - “Used history table → to track changes over time”
 - “Used M:N table → because multiple associations exist”
- Always mention **indexes**:

“Indexed foreign keys and high-query columns like emp_id, date”



FINAL SUMMARY

- ✓ Covers all requirements
- ✓ Scalable design
- ✓ Interview-ready explanation
- ✓ Clean normalization
- ✓ Performance optimized

If you want next level prep, I can:

- 👉 Ask you **interview follow-up questions** (very common ones)
- 👉 Convert this into **MongoDB design**
- 👉 Or give **real queries (SELECT problems)** based on this schema